Advanced RAG Pipeline

What was added to shopnow-rag: LLM as a judge, query expansion and decomposition, cost-aware strategy selection and the adaptive loop - with the call sequence

our function | data / search | OpenAI call | decision / loop | returns

1. What is new

Added step	What it does	File	Extra cost
LLM judge	Checks every answer (context relevant, answer relevant, grounded, score 1-5) before the customer sees it	judge.py	1 call per answer
Query expansion	Rewrites the question in the words the documents use	query_strategies.py	1 cheap call, only if needed
Query decomposition	Splits a multi-part question into sub-questions	query_strategies.py	1 cheap call, only if needed
Free strategy checks	Multi-part? Weak search? Decided with plain Python, before any LLM call	strategy_selector.py	none
Adaptive loop	Answer → judge → if rejected, switch strategy and try again (max 3)	adaptive.py	only on rejection
New UI mode	"Adaptive RAG" + a box showing each attempt and the judge's verdict	ui.py	-

In plain words: The idea in one line: **start cheap, pay more only when a check says it is needed.** Most questions take the direct path and cost the same as before (answer + judge).

2. The adaptive pipeline

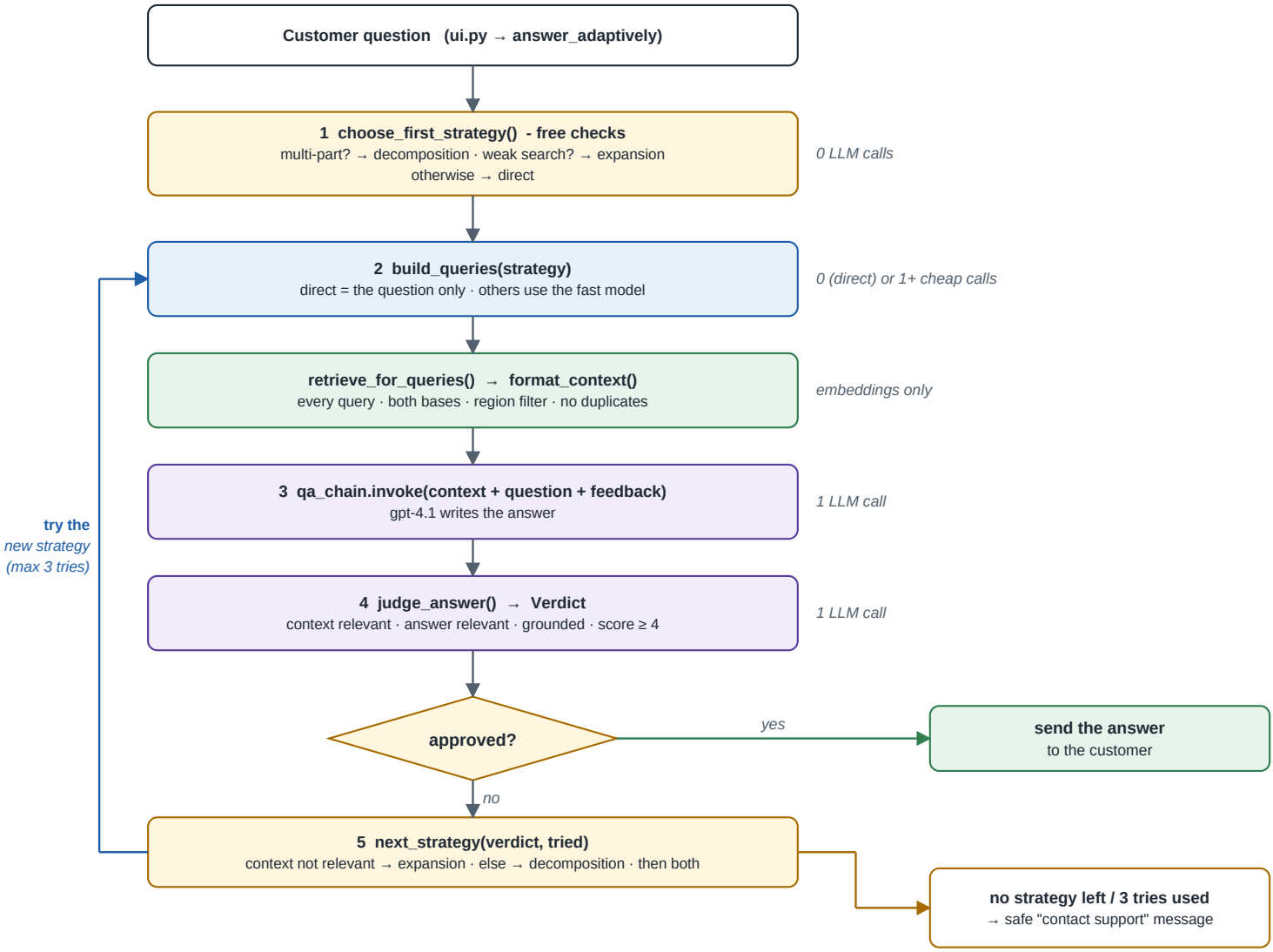


Figure 1 - adaptive.py answer_adaptively(). Right side: what each step costs.

3. The four query strategies

Example question: *"Can I return my smartwatch after 12 days, and how do I get my money back?"*

Strategy	Queries it searches with	LLM calls	Use when
direct	the question as typed	0	short, clear question; good search
expansion	the question + "smartwatch return window", "wearables return days", "refund method"	1 (fast)	customer words differ from document words
decomposition	"What is the return window for smartwatches?" + "How are refunds paid?"	1 (fast)	the question asks several things
decompose+expand	each sub-question + its rewrites	1 + 1 per sub-question (fast)	both problems; last resort

All queries are searched in both bases; the same chunk found twice is kept once (`retrieve_for_queries()`). Then everything continues exactly like the simple chain: context + question → LLM.

4. Using a strategy only when it is needed

When	Check	How (no LLM)	Result
Before answering	is_multi_part()	2+ question marks, a joining phrase ("as well as", "also"…), or > 25 words	decomposition
Before answering	best_distance()	FAISS distance of the closest chunk > WEAK_RETRIEVAL_DISTANCE	expansion
Before answering	neither	-	direct (no extra cost)
After the judge rejects	context not relevant	from the Verdict	expansion → decomposition → both
After the judge rejects	anything else	from the Verdict	decomposition → expansion → both

Situation	LLM calls
Clear question, good search, approved (most questions)	2 (answer + judge) - same as before
Multi-part or weak search, approved	3 (+1 cheap query call)
Rejected once, then approved	about 5
Worst case: 3 attempts, nothing approved	about 8 → safe "contact support" message

Note: Set WEAK_RETRIEVAL_DISTANCE once. Distances depend on the embedding model. Run `python -m app.calibrate`: it prints the best distance for questions the documents answer and for ones they don't. Put a value between the two groups in .env.

5. The LLM judge

Verdict field	Question the judge answers
context_relevant	Do the retrieved chunks contain what is needed to answer?
answer_relevant	Does the answer address what the customer asked?
grounded	Is every fact (number, time limit, condition) in the context?
score (1-5), reason	Overall quality, and one sentence on what is wrong

- **Approved** = all three true and score ≥ JUDGE_MIN_SCORE (4) - `is_approved()`.
- **Adaptive mode**: a rejection switches the query strategy (section 4).
- **Simple chain and agent modes**: `verify_answer()` rewrites once with the judge's reason as feedback; if the context itself is not relevant it sends the safe message right away.
- **An unapproved answer is never shown to the customer.**

6. Call sequence

Read top to bottom in the order the code runs. An indented box is called inside the box above it; the right column is what goes back up to the caller.

6.1 Adaptive RAG mode - call tree

CALL (order · function)	WHERE	RETURNS (back to the caller)
Customer clicks Ask (mode = "Adaptive RAG")	`ui.py 143`	
1 answer_question(question, region, "Adaptive RAG")	`ui.py 64`	→ *(answer, sources, attempts, verdict)*
2 remove_card_numbers · get_llm · get_vector_stores	`ui.py 81-85`	→ *question, llm, stores*
3 create_qa_chain · create_judge · create_expander / decomposer	`ui.py 86-93`	→ *the 4 chains (not run yet)*
4 answer_adaptively(question, stores, qa_chain, judge, …)	`adaptive.py 19`	→ *(answer, attempts, verdict, docs)*
5 choose_first_strategy(question, stores, region)	`selector.py 57`	→ *(strategy, reason)*
6 is_multi_part(question) - plain Python	`selector.py 21`	→ *True / False*
7 best_distance() → similarity_search_with_score	`selector.py 37`	→ *closest distance*
LOOP for each attempt (max 3)	`adaptive.py 42`	
9 build_queries(question, strategy, expander, decomposer)	`strategies.py 51`	→ *list of queries*
10 expander / decomposer.invoke - skipped for direct	`strategies.py 68-77`	→ *QueryList (fast model)*
11 retrieve_for_queries(queries, stores, region)	`strategies.py 81`	→ *unique chunks*
12 retrieve(query, store, region) - per query, per base	`retriever.py 10`	→ *chunks*
13 format_context(documents)	`retriever.py 31`	→ *CONTEXT text*
14 qa_chain.invoke(region, context, question, feedback)	`adaptive.py 49`	→ *answer text*
15 judge_answer(question, context, answer, judge)	`adaptive.py 52`	→ *Verdict*
16 is_approved(verdict)? → return the answer	`adaptive.py 55-56`	→ *approved answer (loop ends)*
17 next_strategy(verdict, tried)	`adaptive.py 59`	→ *next strategy or None*
18 nothing approved → return NOT_FOUND	`adaptive.py 66`	→ *safe message*
19 format_sources · format_attempts · format_verdict	`ui.py 94`	→ *4 text boxes in the UI*

Figure 2 - Adaptive mode. selector = strategy_selector.py, strategies = query_strategies.py.

- **Down**: answer_question → answer_adaptively → choose_first_strategy (free) → loop: build_queries → retrieve_for_queries → format_context → qa_chain → judge.
- **Back up**: approved answer (or NOT_FOUND) + the list of attempts → answer_question → the UI's four boxes.

6.2 Adaptive RAG mode - one question that needs a second try

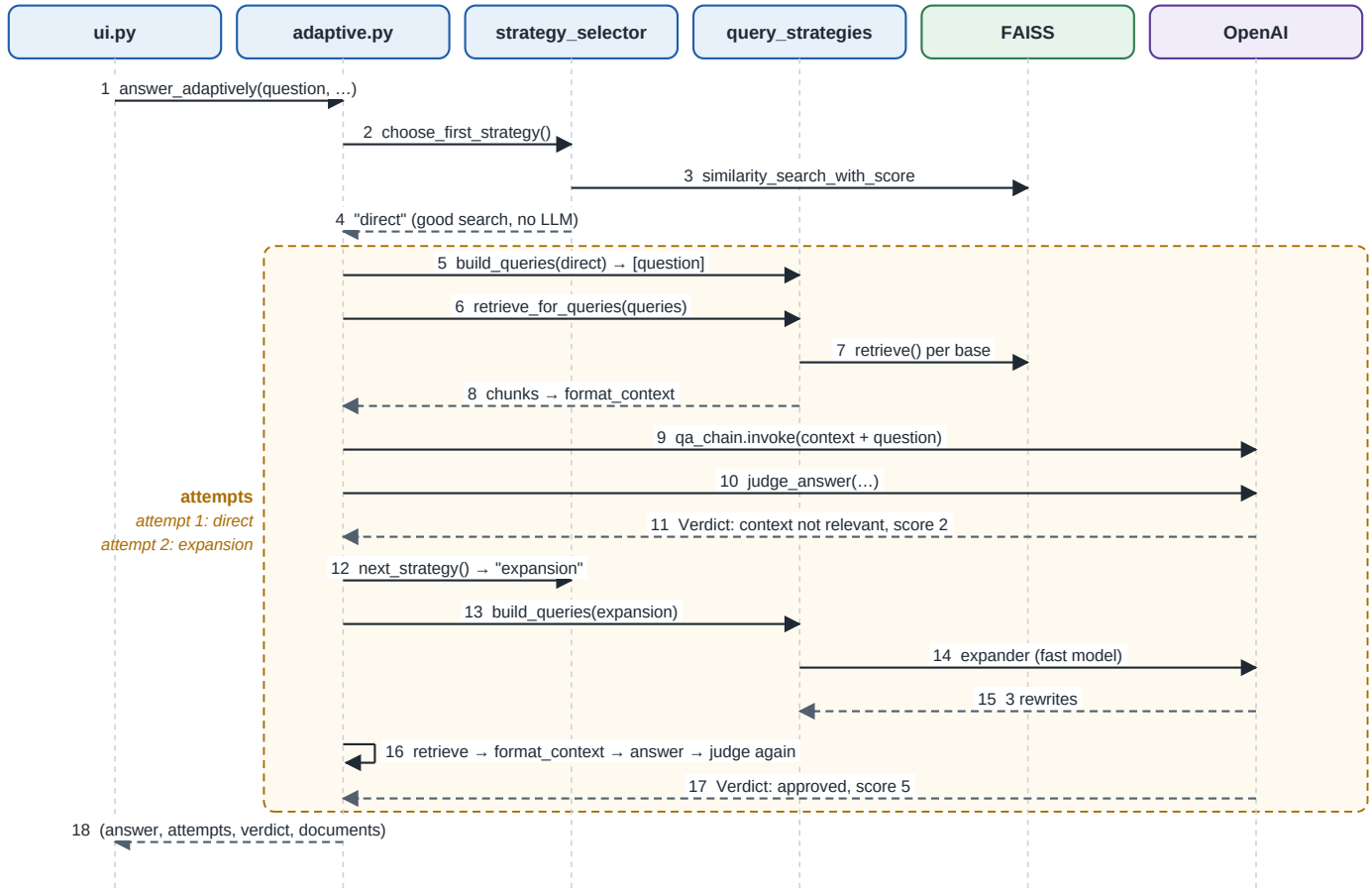


Figure 3 - Attempt 1 (direct) is rejected because the context is not relevant; attempt 2 (expansion) is approved. Dashed = returned value.

6.3 Judge in the simple chain and agent modes - call tree

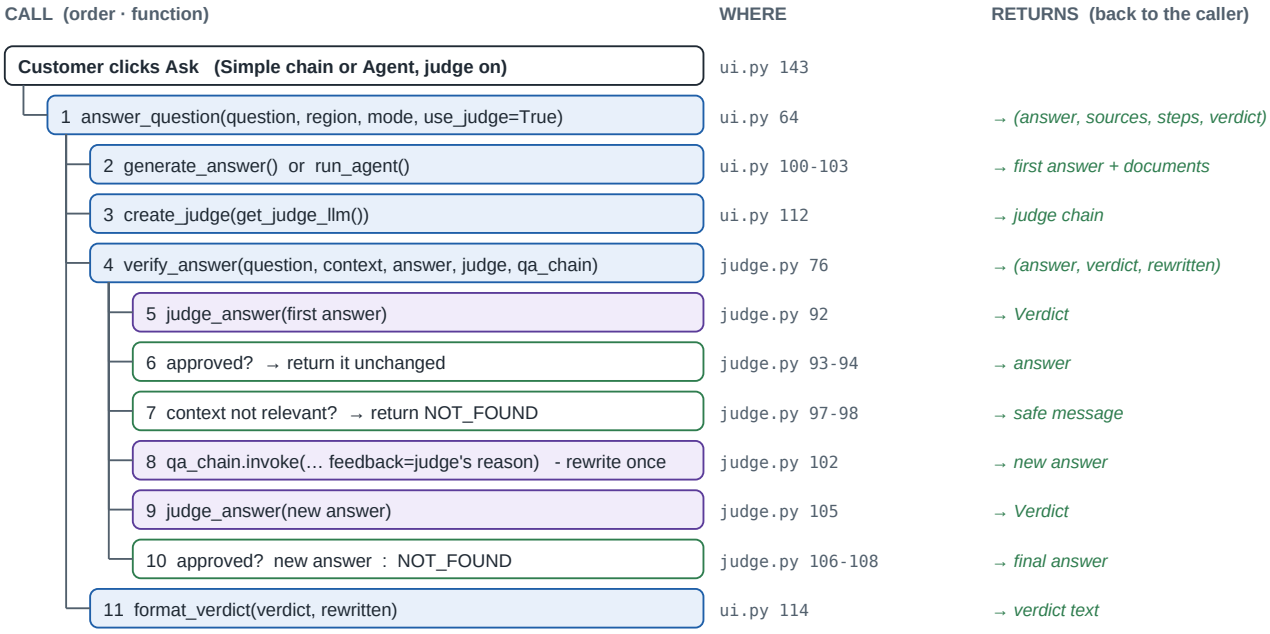


Figure 4 - verify_answer(): check, maybe rewrite once, check again.

7. New Python and LangChain ideas

Idea	Why here	Where
Pydantic BaseModel (Verdict, QueryList)	The data comes from the LLM, so it must be checked: missing field or wrong type = clear error	judge.py, query_strategies.py
with_structured_output(Model)	OpenAI must reply in exactly that shape - no text parsing	create_judge, create_expander
Constants for names (DIRECT, EXPANSION…)	One spelling for each strategy; a typo becomes an import error, not a silent bug	query_strategies.py
dict keyed by chunk text	Remove duplicate chunks, keep the order found	retrieve_for_queries()
list of dicts (attempts)	Simple record of each try, only displayed (a @dataclass would also work if it grows)	adaptive.py
similarity_search_with_score	Gives the distance with each chunk, so a weak search can be detected for free	best_distance()
{feedback} prompt slot	The same answer prompt can rewrite using the judge's reason	prompts.py
RunnableLambda, monkeypatch (tests)	Fake judge / expander and a fixed threshold, so every path is tested offline	tests/

8. Settings, run and test

```
# .env (all optional)
JUDGE_MODEL=gpt-4.1          FAST_MODEL=gpt-4.1-mini          WEAK_RETRIEVAL_DISTANCE=1.0
# config.py
JUDGE_MIN_SCORE = 4          MAX_ATTEMPTS = 3                 MULTI_PART_MIN_WORDS = 25

python -m app.ingest        # build the indexes
python -m app.calibrate     # pick WEAK_RETRIEVAL_DISTANCE
python -m app.ui            # choose mode "Adaptive RAG"
pytest                      # 27 offline tests (judge paths, strategies, escalation, cost)
```